Voor jouw apotheek

Svelte Fundamentals

Module – More `$props()`

Peter Kassenaar – info@kassenaar.com

eduvision
opleiding & training

More on $props()

Using the `$props()` rune to the max

The `$props()` rune

- Mostly: using destructuring when sending props
- Props can have a default value

```
let {
    message,
    defaultMessage = 'Please provide a message'
} = $props();
```

```
<!--    Using destructured props-->
<h2>Message: <code>{ message }</code></h2>
<h2>Default Message: <code>{ defaultMessage }</code></h2>
```

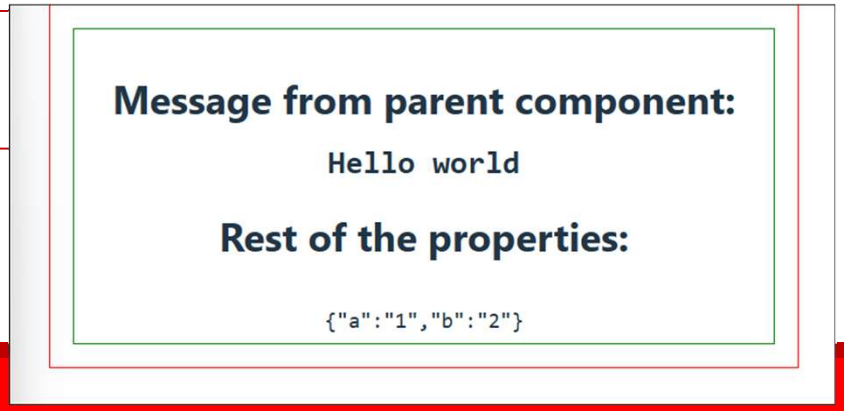
Using the rest operator `...rest`

- Use the rest operator … to get (literally) ‘the rest’ of the properties
- For instance:

```
<DefaultProps message="Hello world" a="1" b="2"/>
```

```
let { message, ...rest} = $props();
```

```
<h2>Rest of the properties:</h2>
<code>{ JSON.stringify(rest) }</code>
```



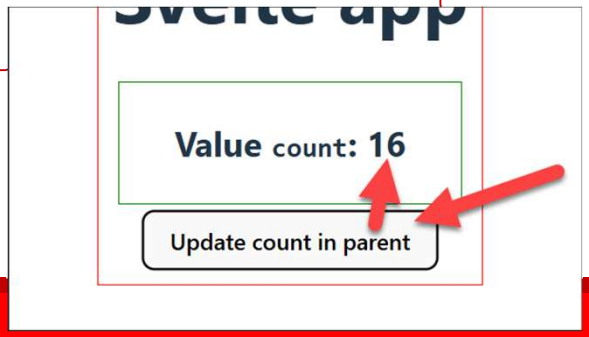
Updating properties

- Add them to `$state()` in parent component
- Pass to child component:
  - State changes in Parent, will propagate to Child

```
let count = $state(10);
```

```
button onclick={()=>count++}>Update count in parent</button>
```

```
<!--    Prop is updated in the parent component-->
<h2>Value <code>count</code>: {count}</h2>
```



Mutating props

- Convention: don’t mutate props in the child!
- Create pure view components
- Mutations should be propagated back to the parent using `$bindable()`

https://svelte.dev/docs/svelte/$props

Child.svelte

JS TS

```
<script lang="ts">
  let { object = { count: 0 } } = $props();
</script>

<button onclick={() => {
  // has no effect if the fallback value is used
  object.count += 1
}}>
  clicks: {object.count}
</button>
```

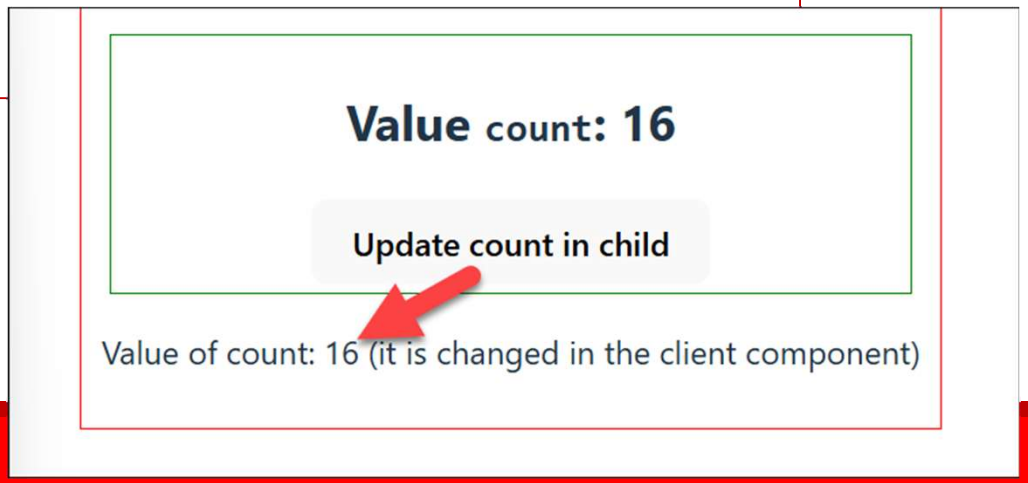
In summary: don't mutate props. Either use callback props to communicate changes, or — if parent and child should share the same object — use the `$bindable` rune.

Using `$bindable()`

- When you want changes in props to be reflected in parent component, use `$bindable()` and `bind:`

```
<BindableProps bind:count={count}/>
<p>Value of count: {count} (changed in the client component)</p>
```

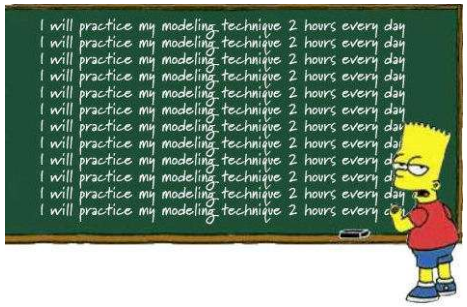
```
let {
    // bindable prop, changes are propagated to the parent.
    count = $bindable(),
} = $props();
```



Workshop

- Create a component `UserCard.svelte` that accepts the props `name` and `role`. Display them in a small card in the UI
  - Let the parent component use/call this component and pass the correct props
- Enhance your component by adding a prop `status` with default value `offline`.
  - Change the value by sending a prop
- Capture the remaining props with `…rest`. Apply them in the parent component
- Make the prop `status` `$bindable()`
  - Add a button to toggle the status: onine=green, offline=red
- Basic example: ./160-more-props

```svelte
<UserCard
    name="Alice"
    role="Admin"
    class="highlight"
    data-id="123"
    bind:status
/>
```



Checkpoint

- You know about options for passing props
- You can give props a default value
- You can create an interface for props
- You are able to create bindable props that are changed in the child component